

PROJET

Ruche connectée : conception et réalisation de la Carte Lora

Lycée Les Persévérants

BTS Cybersécurité Informatique et réseaux, Électronique
2023 – 2025

MUSQUET Allan

SOMMAIRE

Remerciements.....	3
Diagramme de Gant.....	4
Introduction.....	5
1. Dossier d'étude préliminaire.....	6
a. Diagramme de définition des blocs.....	6
b. diagramme de bloc interne.....	7
2. Dossier d'étude de la carte.....	7
a. Critères de choix du microcontrôleur ESP32.....	7
b. Critères de choix du transceiver CAN MCP2551.....	7
c. Critères de choix du module Lora.....	7
3. Dossier de réalisation de la carte.....	7
a. Schéma structurel de la carte.....	7
b. Typon.....	9
c. Schéma d'implantation.....	11
d. Nomenclature.....	11
4. Dossier de procédure et de test de la carte.....	11
a. Soudure et schéma de câblage.....	11
b. Les différents tests à réaliser.....	13
c. Cahier de recette.....	13
d. Code source.....	13
e. Résultats des tests.....	13
Conclusion.....	14
Glossaire et liste des abréviations.....	15
Annexes.....	17

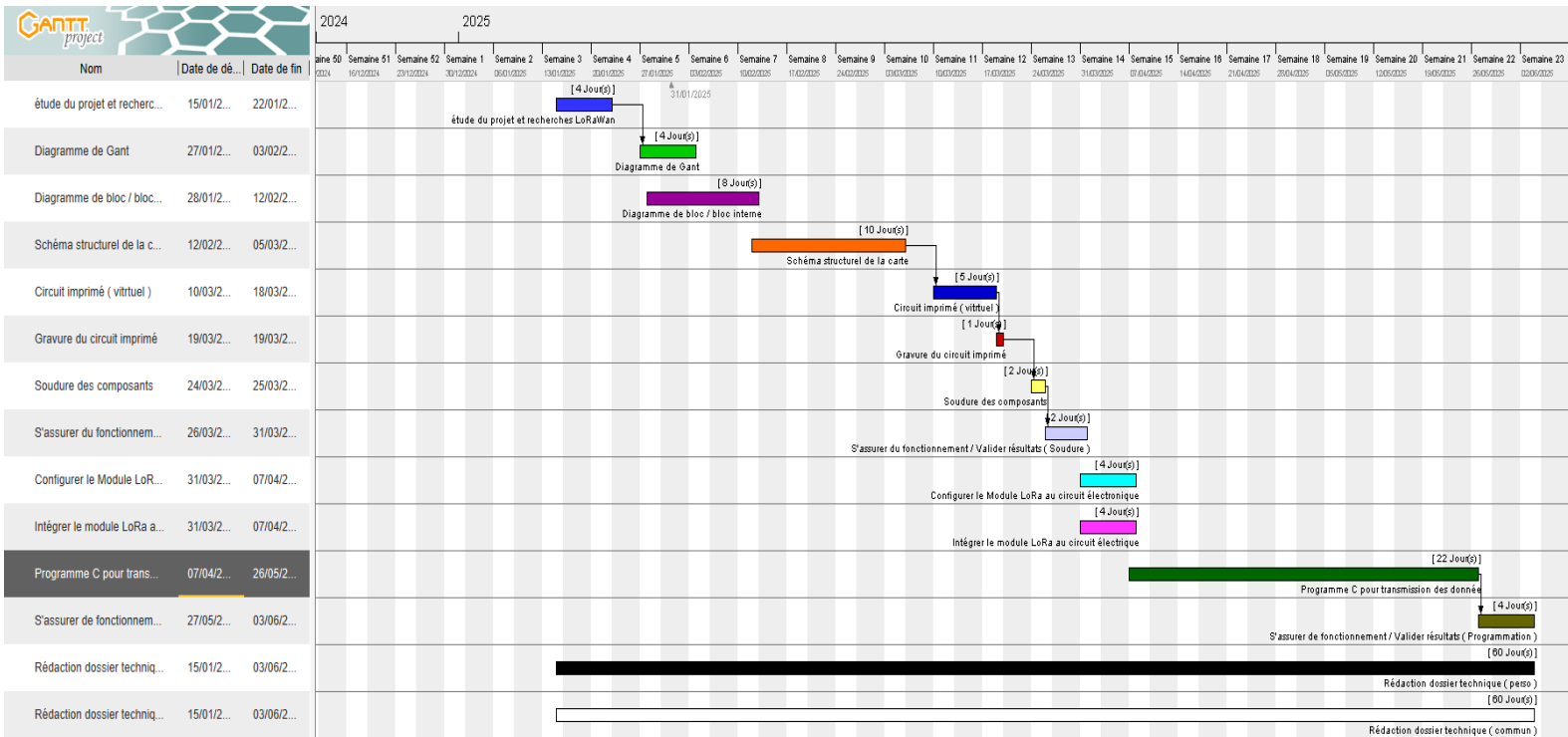
Remerciements

Je tiens à exprimer mes remerciements au Lycée Les Persévérants de Guadeloupe pour l'accompagnement durant mon la réalisation de mon projet, qui s'est déroulé du 5 Janvier 2025 au 15 juin 2025 au sein du BTS CIEL (Cybersécurité Informatique et réseaux, Électronique).

Je tiens à remercier tout particulièrement mes professeurs, Mr Thomas Bourdeau, et Madame Landre pour leur encadrement, leurs conseils et leur disponibilité tout au long de cette expérience. Leur accompagnement et pédagogie ont été des atouts majeurs dans l'apprentissage de nouvelles compétences.

Diagramme de Gant

Je décide de créer un diagramme de Gant pour visualiser chaque étape que je vais devoir réaliser pour assurer une bonne compréhension et continuité du projet tout en essayant de rester dans les délais impartis. Celui-ci évoluera probablement au fil du temps suivant les différents facteurs rencontrés durant la réalisation du projet (en meilleure qualité en annexe).



Introduction

LoRaWAN = " Long Range Wide Area Network " est un protocole de communication sans fil utilisée pour connecter des objets à Internet, de manière simple et économique, sur de longues distances. Le BUS CAN envoie les données des capteurs à un module LoRa (une puce de communication). Celui-ci convertit les données en un signal radio longue portée et basse consommation. Ce signal est envoyé à une passerelle LoRaWAN. Celle-ci reçoit les données envoyées par la ruche et les transmet via Internet à un serveur. L'apiculteur peut alors consulter les mesures sur une interface en ligne.

2.2. Qu'est ce que LoRa ?

LoRa, c'est une technologie sans fil qui permet à des objets (comme des capteurs) d'envoyer de petites données sur de très longues distances (jusqu'à 20 km).

LoRa utilise la modulation Chirp Spread Spectrum (CSS), qui permet une transmission robuste, même dans des environnements avec des interférences. Les données sont envoyées via des émetteurs LoRa vers des récepteurs ou des passerelles, puis transférées à un serveur pour traitement.

2.3. A quoi sert le système LoRaWan dans le projet ?

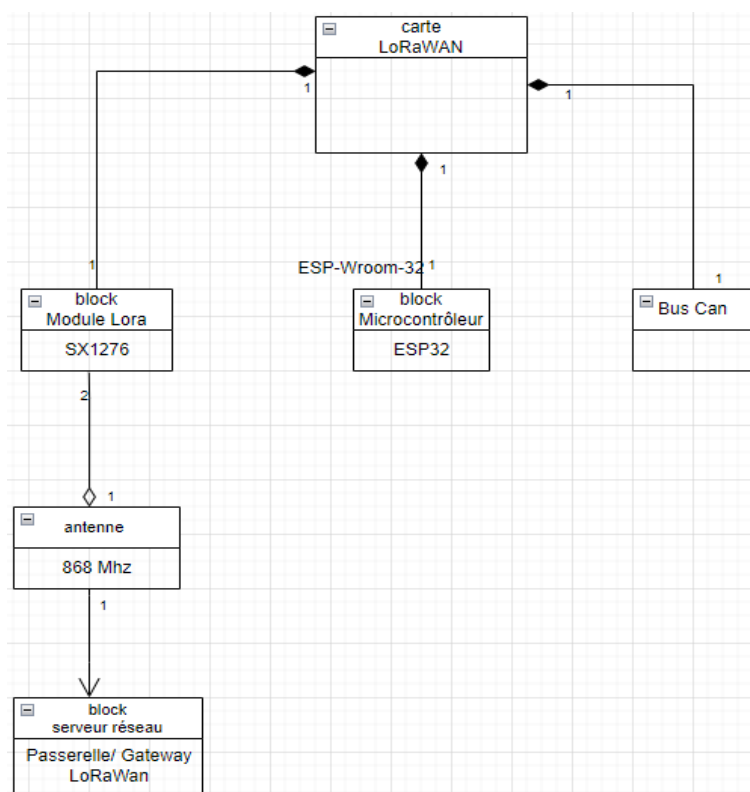
LoRaWAN servira principalement à transmettre les données des capteurs (poids, température et humidité) à l'apiculteur sur un serveur, même si la ruche est située dans une zone éloignée sans WiFi ni réseau mobile classique, tout en consommant très peu d'énergie. C'est la clé pour un suivi efficace et autonome des ruches connectées .

1. Dossier d'étude préliminaire

Dans le cadre de la phase d'analyse du système, il est essentiel de modéliser les différents composants et leur organisation afin de mieux comprendre l'architecture globale du projet. Pour cela, l'utilisation des diagrammes de blocs et de bloc interne permettent de représenter de manière synthétique les éléments techniques, leurs interactions et les responsabilités associées.

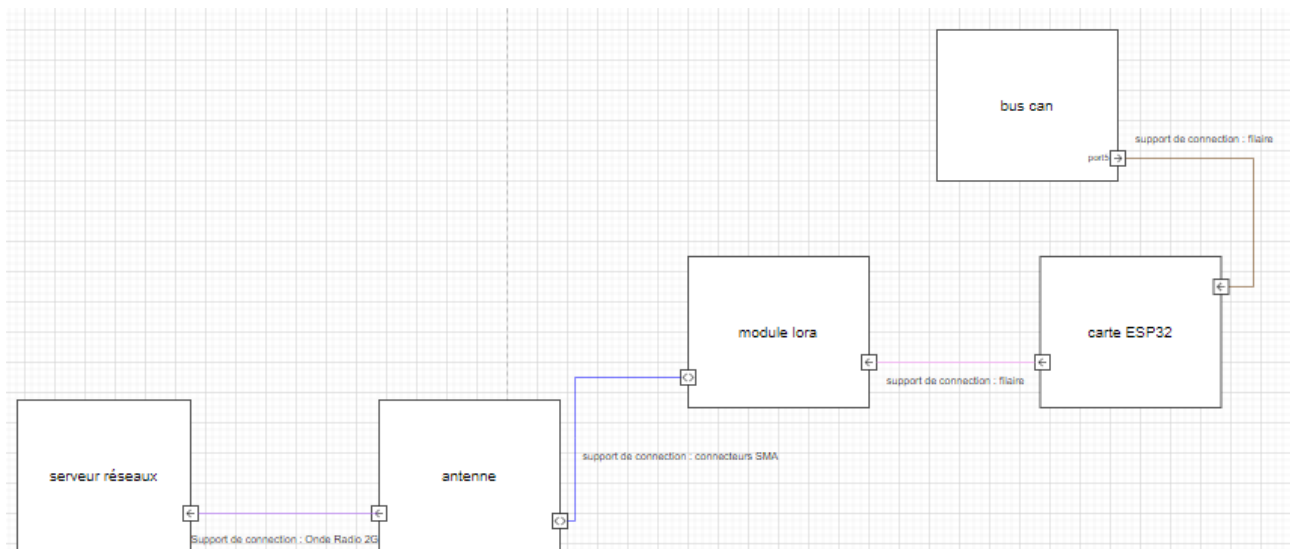
a. Diagramme de définition des blocs

Je décide de créer un diagramme de bloc pour définir la structure du système et les relations entre les composants. Celui ci montre les éléments du système (Wemos D1 R32, LoRa, Bus CAN, etc.), décrit les relations de composition et d'agrégation ("Carte LoRaWAN contient un module LoRa"), et aide à identifier les responsabilités de chaque composant.



b. diagramme de blocs internes

Et enfin, je décide de créer un diagramme de bloc interne pour montrer les connexions et les flux d'information entre les composants. Il a une vue dynamique qui détaille comment les blocs communiquent entre eux.



2. Dossier d'étude de la carte

a. Critères de choix du microcontrôleur ESP32 :

L'ESP32 présente plusieurs avantages majeurs, Sur le plan matériel, elle embarque un processeur double cœur cadencé à 240 MHz, Cela permet de traiter plus rapidement les différentes tâches du projet, notamment la lecture des capteurs, la gestion du bus CAN, et la transmission des données par LoRa.

Au niveau de la mémoire, l'ESP32 offre 520 Ko de RAM et peut aller jusqu'à 16 Mo de mémoire flash. Cette capacité mémoire plus importante permet de stocker davantage de données et de gérer des programmes plus complexes, tout en conservant une bonne réactivité. Un autre avantage est l'intégration native de la connectivité WiFi et Bluetooth dans l'ESP32. Bien que le projet utilise principalement le protocole LoRaWAN pour la transmission des données, ces connectivités supplémentaires sont très utiles lors des phases de test, ou de configuration.

L'ESP 32 reste aussi très peu coûteuse tout en étant compatible avec l'environnement de développement Arduino IDE, ce qui facilite grandement la transition sans nécessiter de revoir entièrement le code source.

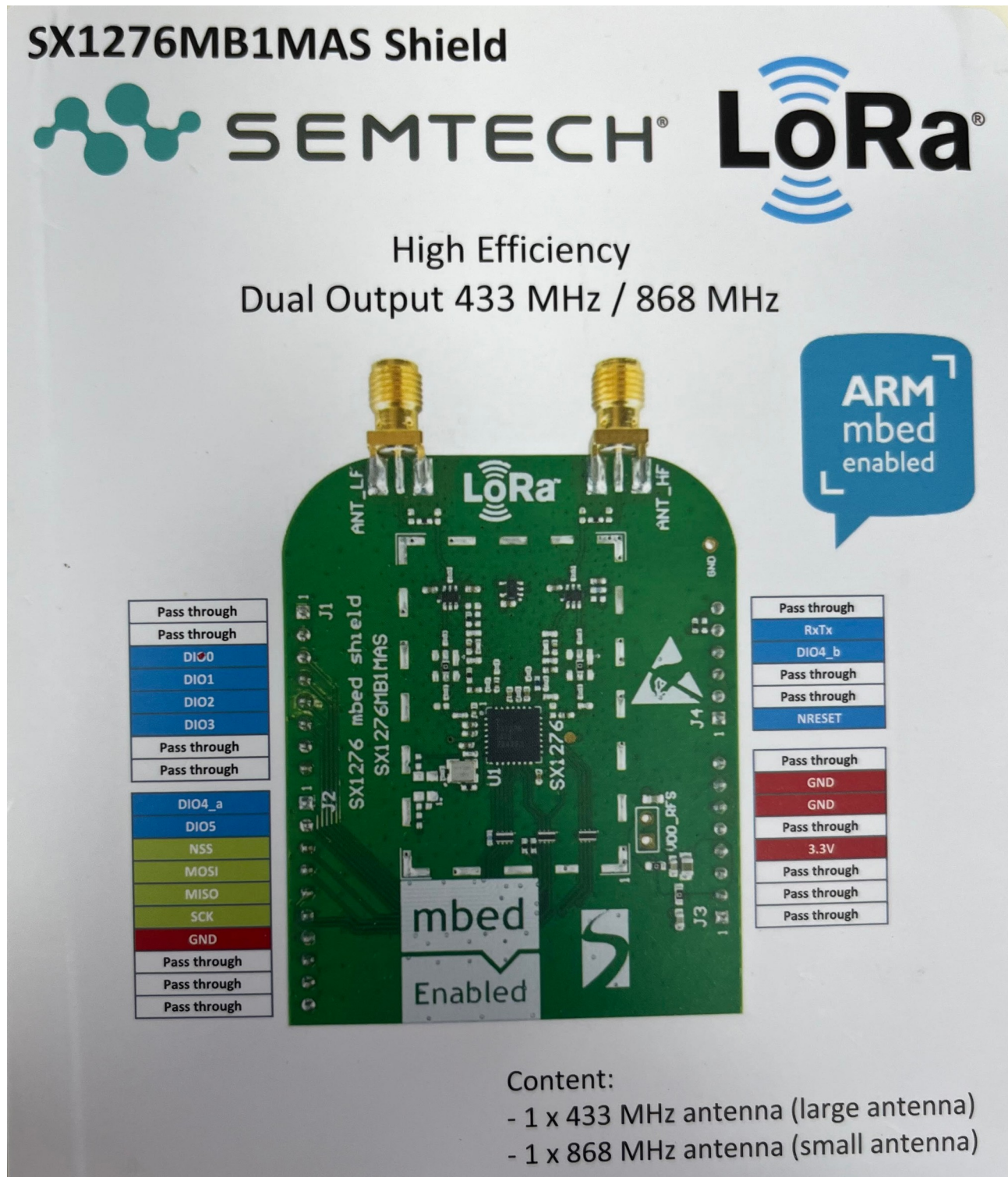
b. Critères de choix du transceiver CAN MCP2551 :

j'ai opté pour le composant MCP2551, un transceiver CAN (Controller Area Network). Il assure l'interface entre le bus CAN, et le microcontrôleur (ESP32). Le choix du MCP2551 se justifie par plusieurs avantages comme la compatibilité avec la norme CAN 2.0, la Robustesse du signal, la protection intégrée, et enfin la faible consommation et un encombrement réduit.

c. Critères de choix du module Lora :

Le SX1276 peut atteindre jusqu'à 10 km en champ libre et plusieurs kilomètres en milieu semi-urbain, ce qui est idéal pour couvrir la zone autour des ruches, souvent isolée ou rurale. Pour un système autonome en énergie (selon le cahier des charges), le SX1276 consomme très peu, surtout en mode veille ou lors de transmissions courtes et espacées (ici : toutes les heures). Cela prolonge l'autonomie de la ruche. Il permet une excellente qualité de réception même avec un signal faible. Le module SX1276 communique en SPI, facilement gérable par l'ESP32 choisi pour le projet. Le coût faible du SX1276 rend ce module facilement accessible et en fait un bon rapport qualités prix.

Image du module LoRa SX1276 :



3. Dossier de réalisation de la carte

a. Schéma structurel de la carte

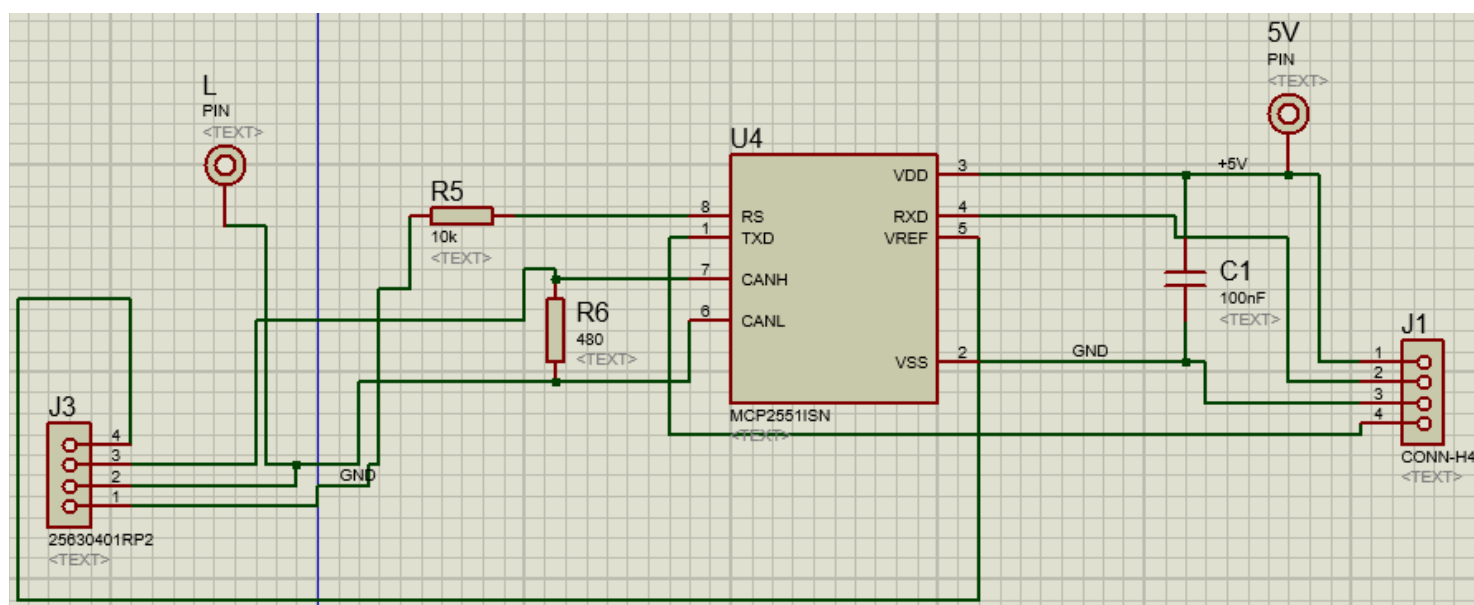
L'ensemble de ce schéma constitue l'interface physique entre le microcontrôleur et le bus CAN, assurant à la fois :

la conversion des niveaux logiques (grâce au MCP2551),

la stabilité électrique (grâce à R5 et C1),

et l'intégrité du signal sur le réseau CAN (grâce à R6).

Ce module peut ainsi être intégré dans un système embarqué, tel que la ruche connectée, pour transmettre les mesures de poids, d'humidité ou de température via le protocole CAN avant envoi par LoRa.



- Interface CAN (U4 - MCP2551). Le MCP2551 est un transceiver pour bus CAN (Controller Area Network), un protocole de communication largement utilisé dans l'industrie ou encore les systèmes embarqués. Il permet la communication fiable via le bus CAN via le réseau CAN.
- Il convertit des signaux logiques provenant du microcontrôleur en signaux compatibles avec le bus CAN.
- Il réceptionne et l'émission de données via les lignes différentielles CANH (CAN High) et CANL (CAN Low).

Les données du bus CAN sont captées via CANH et CANL, puis il reconstruit le signal logique et l'envoie au microcontrôleur via la broche RXD.

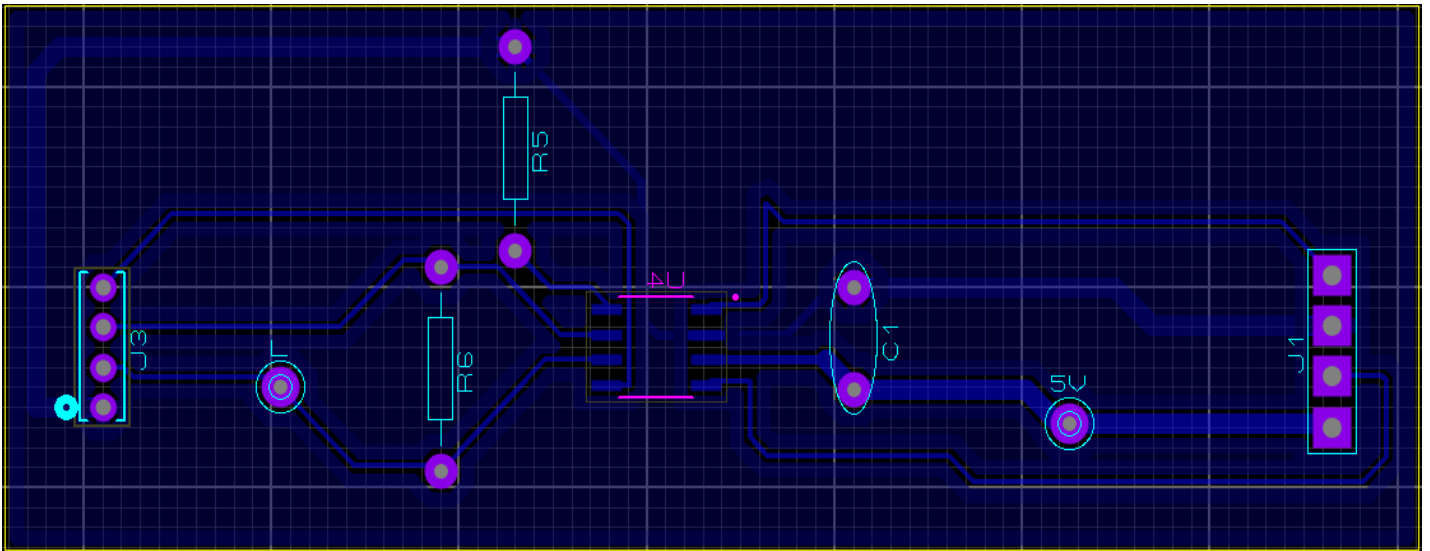
- J1 (Connecteur) : Ce connecteur 4 broches permet l'interconnexion avec les lignes du bus CAN. Les deux lignes différentielles CANH et CANL y sont reliées, assurant la transmission et la réception des données sur le réseau CAN.
- J3 (Connecteur d'entrée) : Il reçoit les signaux provenant d'un microcontrôleur externe (ESP 32). La broche d'entrée est connectée via une résistance à la broche TXD du MCP2551.
- R5 (Résistance de tirage de 10k Ω) : Placée entre la ligne TXD et l'alimentation +5V, elle assure un état logique stable lorsqu'aucun signal n'est activement transmis.
- R6 (Résistance de terminaison 480 Ω) : Connectée entre CANH et CANL, elle assure l'adaptation d'impédance du bus CAN. Cette résistance est cruciale pour éviter les réflexions de signal sur le bus et garantir une communication fiable.
- C1 (Condensateur de découplage de 100nF) : Placé entre VDD et GND, il filtre les parasites et stabilise la tension d'alimentation du MCP2551, ce qui protège le circuit contre les perturbations.
- L'alimentation 5V : Fourni via la broche supérieure droite, elle alimente l'ensemble du circuit, notamment le MCP2551 et ses composants passifs.

b. Typon

Le typon est le plan de fabrication du circuit imprimé. Il montre :

- ◆ les pistes de cuivre qui relient les composants
- ◆ plusieurs couches : face cuivre, sérigraphie, masque de soudure, etc.

Ce typon représente virtuellement la version imprimée du circuit électronique présenté précédemment.

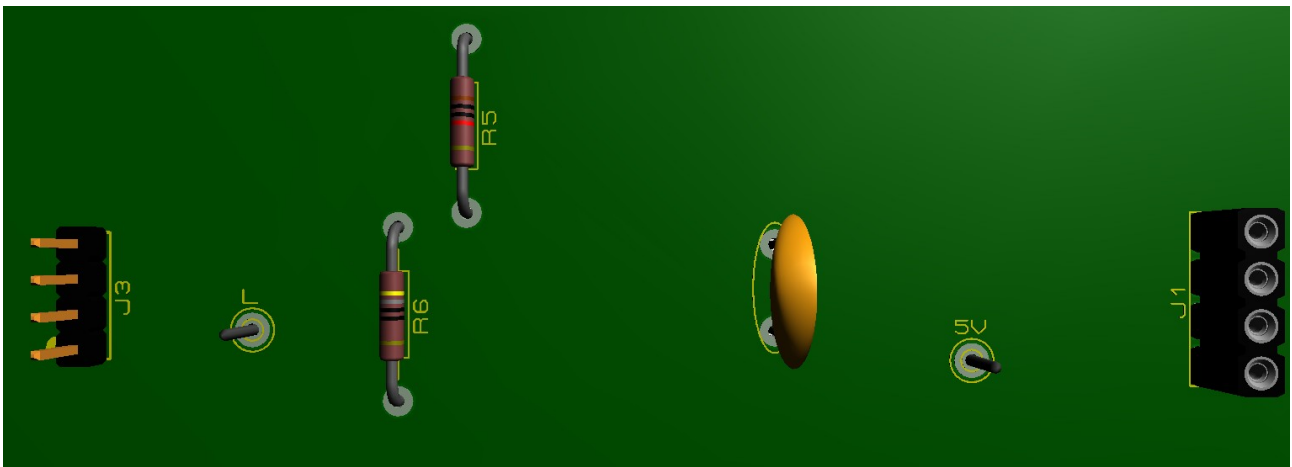


Le MCP2551, situé au centre du PCB, est connecté aux lignes TXD et RXD, qui permettent l'échange de données avec un microcontrôleur, et aux lignes CANH et CANL, qui assurent la transmission des signaux sur le bus CAN. L'alimentation est fournie en +5V, avec une masse bien distribuée pour la stabilité du circuit.

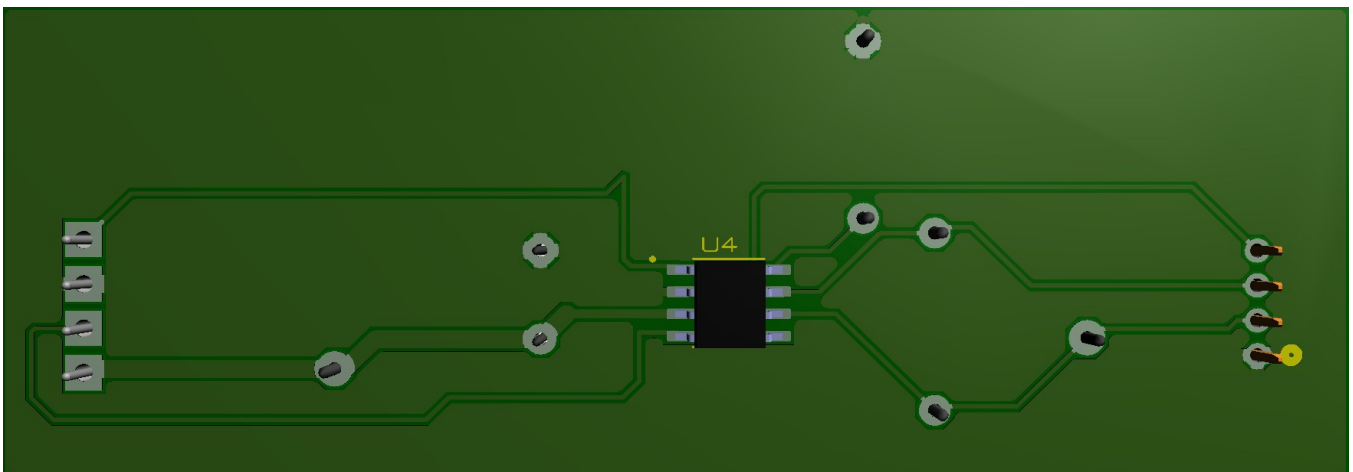
Deux connecteurs, J2 et J3, sont présents pour relier le circuit au bus CAN et à l'alimentation. Le circuit inclut également des composants essentiels : R5 (1k Ω) pour limiter le courant, R6 (480 Ω) en tant que résistances de terminaison du bus CAN, et le condensateur de découplage C1 (100nF) pour filtrer et stabiliser l'alimentation.

- Visualisation 3D du circuit imprimé virtuel (Face haut) ; (Face bas) :

Face haut

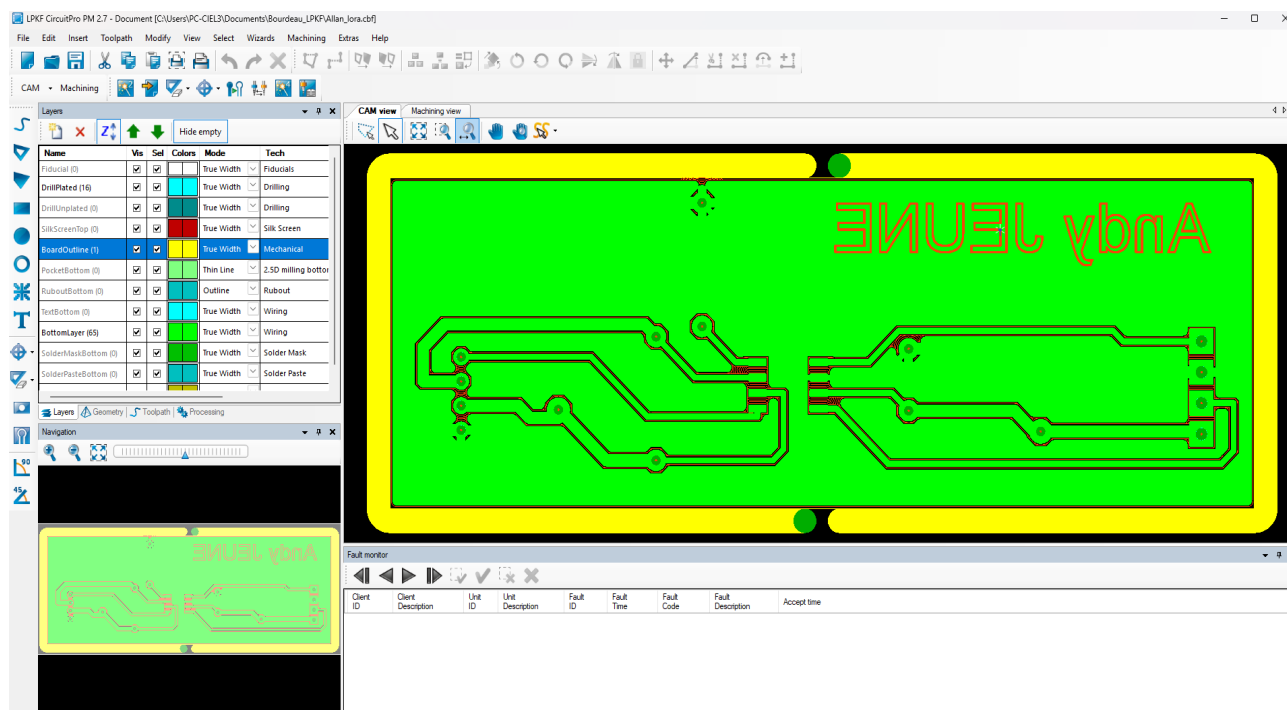


Face bas



c. Schéma d'implantation

C'est le plan d'implantation qui montre l'emplacement physique exact de chaque composant électronique sur le PCB.



d. Nomenclature

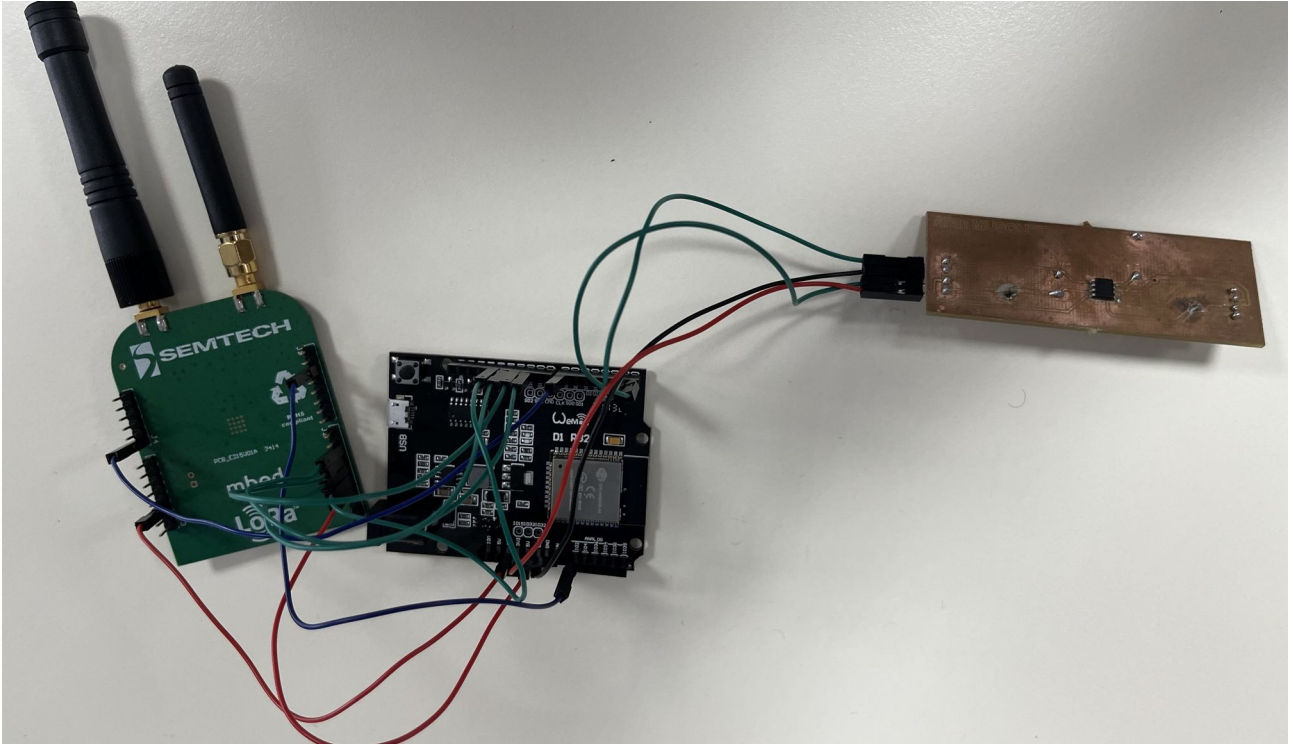
La nomenclature représente une liste détaillée de tous les composants utilisés dans un projet électronique, avec leurs références, quantités, valeurs, empreintes. Elle est donnée dans le tableau ci-contre :

Composants	Quantité	Références	Valeur
Résistance	02	R3, R4	480 ohms, 10k
Condensateur	01	C1	100 nf
Interface CAN	01	U4 ?	MCP2551
Connecteurs	02	J1, J3..	CONN-SIL4

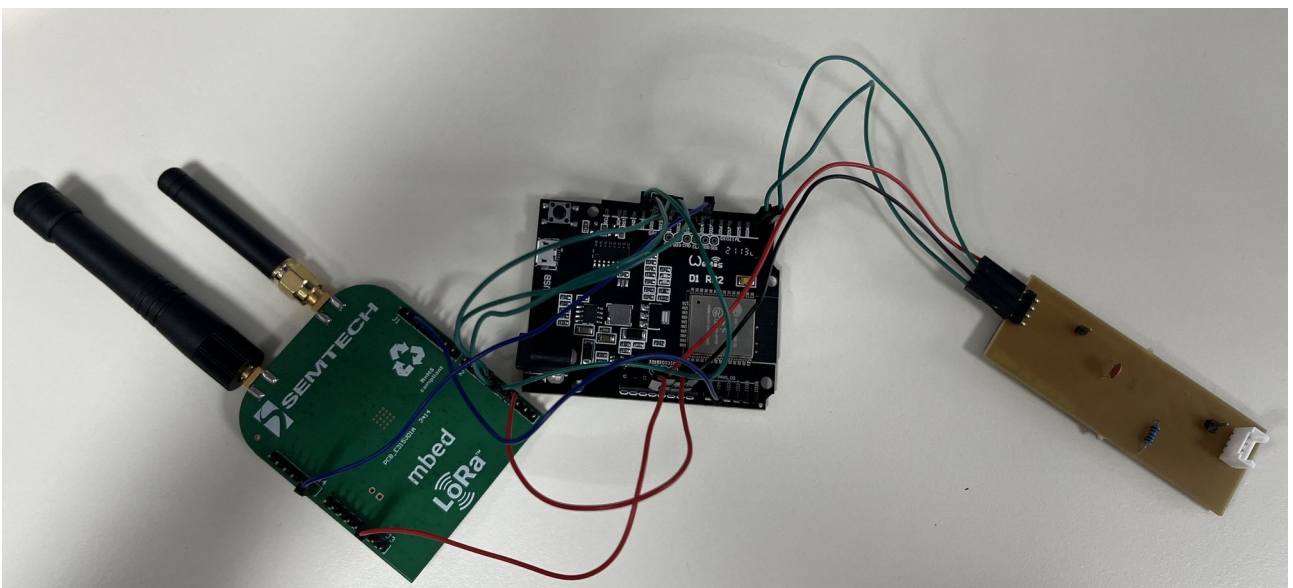
4. Dossier de procédure et de test de la carte

a. Soudure et schéma de câblage

Soudure et schéma de câblage recto :



Soudure et schéma de câblage verso :



- Gravure de la carte sous fraiseuse (physique)

Pour la gravure de la carte Lora , j'ai utilisé une fraiseuse LPKF Laser & Electronics, adaptée à la gravure et au perçage de circuits imprimés. La machine a été pilotée via le logiciel CircuitPro PM 2.7, permettant de configurer les opérations de gravure, perçage et découpe en tenant compte des tolérances et dimensions.

Outils et consommables utilisés :

- Plaque en cuivre : support du circuit imprimé.
- Overlay en carton : Support de gravure.
- Forets en spirale (1,0 mm / 1,20 mm / 1,50 mm / 2,00 mm) : pré-perçages pour assurer précision et éviter les déformations du cuivre ou du foret (couleur vert sur la photo ci-dessous).
- Forets cutter (1,0 mm / 1,5 mm) : perçages définitifs adaptés aux composants traversants (couleur rouge sur la photo ci-dessous).
- Foret de 2,0 mm : découpe des contours de la carte, et de la signature de celle-ci (couleur jaune sur la photo ci-dessous).



Processus réalisé :

1. Pré-perçage des points de perçage avec les forets en spirale.
2. Perçage définitif avec les forets cutter pour des trous, et des tracés précis.
3. Découpe du contour, et la signature (nom, prénom) de la carte avec le foret de 2,0 mm selon les tracés du logiciel.

Remarques :

Toutes les étapes ont été réalisées en respectant les consignes de sécurité et le traitement des déchets.

Une attention a été portée à la vérification des dimensions et à la qualité des perçages pour une intégration sans défaut des composants et du module LoRa.

Dans le cadre de l'assemblage de la carte Lora, j'ai réalisé la soudure des composants. Pour cela, j'ai utilisé un fer à souder, de l'étain-plomb pour assurer des soudures durables et de qualité, et du flux pour améliorer la capillarité et brasures propres.

Afin de garder une efficacité, et une propreté du fer à souder, j'ai régulièrement utilisé de la paille de fer durant les opérations.

Les composants soudés sont les suivants :

- Résistances de 10 k Ω et de 480 Ω , servant à la polarisation d'une ligne de communication.
- Condensateur, utilisé pour le découplage de l'alimentation.
- Circuit intégré MCP2551, transceiver CAN pour la communication entre le microcontrôleur et le bus CAN.
- Pins de connexion :
 - l'alimentation en 5 V,
 - la connexion au réseau CAN L,
 - d'autres liaisons nécessaires à l'interface de la carte avec le système global.

Cette phase de soudure a été réalisée avec soin pour obtenir des soudures efficaces, sans excès d'étain et sans court-circuit. Bien sur, des tests de bon fonctionnements seront réalisés pour assurer un bon fonctionnement lors de l'utilisation de la carte.

b. Les différents tests à réaliser

Les différents tests à réaliser sur notre carte lora sont :

- Tension d'alimentation : vérifier que le module lora reçoit la tension de 3,3V
- Test de continuité
- Test Lora point-à-point : tester un lien direct (émission et réception) entre 02 modules sans gateway
- Test de communication du bus CAN : récupération des données (Température/humidité)
- Test Lorawan : s'assurer que la carte lora connectée à une passerelle lorawan (TTN) transmet un paquet.

c. Cahier de recette

Il s'agit de vérifier que la carte lora fonctionne conformément aux spécifications du cahier des charge. Il est résumé dans le tableau ci-contre :

Test No	But	Conformité
1	Tension d'alimentation	oui
2	Test de continuité	oui
3	Test Lora point-à-point	non
4	Test Communication Bus CAN	non
5	Test Lorawan	non

d. Code source

```
#include "driver/gpio.h"
#include "driver/twai.h"

// Configure message to transmit
twai_message_t message = {
    // Message type and format settings
    .extd = 0,           // Standard vs extended format
    .rtr = 0,           // Data vs RTR frame
    .ss = 0,            // Whether the message is single shot (i.e., does not repeat on error)
    .self = 0,          // Whether the message is a self reception request (loopback)
    .dlc_non_comp = 0,  // DLC is less than 8
    // Message ID and payload
    .identifier = 0xAAAA,
    .data_length_code = 4,
    .data = {50, 50, 50, 50},
};

uint32_t alerts;

void setup()
{
    // Initialize configuration structures using macro initializers
    twai_general_config_t g_config = TWAI_GENERAL_CONFIG_DEFAULT(GPIO_NUM_21, GPIO_NUM_22, TWAI_MODE_NO_ACK);
    twai_timing_config_t t_config = TWAI_TIMING_CONFIG_25KBITS();
    twai_filter_config_t f_config = TWAI_FILTER_CONFIG_ACCEPT_ALL();

    // Install TWAI driver
    if (twai_driver_install(&g_config, &t_config, &f_config) == ESP_OK) {
        printf("Driver installed\n");
    } else {
        printf("Failed to install driver\n");
        return;
    }

    // Start TWAI driver
    if (twai_start() == ESP_OK) {
        printf("Driver started\n");
    } else {
        printf("Failed to start driver\n");
        return;
    }

    // Activation des alertes (version compatible avec toutes les versions)
    twai_reconfigure_alerts(TWAI_ALERT_ALL, NULL);
}

void printCANStatus() {
    twai_status_info_t status;

    if (twai_get_status_info(&status) == ESP_OK) {
        printf("📡 État du contrôleur CAN :\n");
        printf("   TX erreurs      : %d\n", status.tx_error_counter);
        printf("   RX erreurs      : %d\n", status.rx_error_counter);
        printf("   État du bus     : %s\n",
            status.state == TWAI_STATE_BUS_OFF ? "BUS OFF" :
            status.state == TWAI_STATE_RUNNING ? "RUNNING" :
            status.state == TWAI_STATE_STOPPED ? "STOPPED" :
            status.state == TWAI_STATE_RECOVERING ? "RECOVERING" : "INCONNU");
    } else {
        printf("❌ Impossible de lire l'état du CAN");
    }
}

void loop() {
    // Queue message for transmission
    if (twai_transmit(&message, pdMS_TO_TICKS(10000)) == ESP_OK) {
        printf("Message queued for transmission\n");
    } else {
        printf("Failed to queue message for transmission\n");
    }
    delay(1000);
    printCANStatus();

    if (twai_read_alerts(&alerts, pdMS_TO_TICKS(10000)) == ESP_OK) {
        // Affichage de chaque alerte détectée
        if (alerts & TWAI_ALERT_TX_SUCCESS)
            printf("✅ TWAI_ALERT_TX_SUCCESS\n");

        if (alerts & TWAI_ALERT_TX_FAILED)
            printf("❌ TWAI_ALERT_TX_FAILED\n");

        if (alerts & TWAI_ALERT_RX_DATA)
            printf("📄 TWAI_ALERT_RX_DATA\n");

        if (alerts & TWAI_ALERT_RX_QUEUE_FULL)
            printf("❗ TWAI_ALERT_RX_QUEUE_FULL\n");

        if (alerts & TWAI_ALERT_ERR_PASS)
            printf("⚠️ TWAI_ALERT_ERR_PASS\n");

        if (alerts & TWAI_ALERT_BUS_OFF)
            printf("🛑 TWAI_ALERT_BUS_OFF\n");

        if (alerts & TWAI_ALERT_ARB_LOST)
            printf("⚠️ TWAI_ALERT_ARB_LOST\n");
    }
}
```

Ce programme pour ESP32 initialise le bus CAN (TWAI), prépare un message contenant 4 octets de données, et l'envoie toutes les secondes. Il affiche également l'état du bus (erreurs, fonctionnement) et lit les alertes comme les succès ou les erreurs de transmission. Ce code sert à tester et surveiller la communication CAN, utile dans le cadre de la ruche connectée, pour vérifier que les messages circulent correctement entre les modules.

e. Résultats des tests

Les tests électroniques effectués sur la carte PCB se sont révélés concluants : aucun court-circuit ni défaut de soudure n'a été détecté (vu grâce au multimètre), et les signaux des broches TXD et RXD observés à l'oscilloscope sont conformes aux attentes.

En revanche, les essais de communication entre les cartes PCB à l'aide du programme en langage C n'ont pas abouti. Plusieurs dysfonctionnements ont été relevés, notamment lors de l'initialisation de la carte ESP32 et lors de la transmission des données entre celles-ci.

Conclusion

Se projet de ruche connectée (partie LoRaWan) m'a permis d'acquérir des nombreuses compétences dans plusieurs domaines (études de documents techniques, création d'une carte électronique PCB, soudure de composants électroniques, réalisations de tests de fonctionnements, programmations C, ainsi que la rédactions de dossier et support technique.). La mise en œuvre du module LoRaWAN a permis de transmettre les données environnementales de la ruche vers une passerelle LoRaWAN. Cette technologie longue portée, basse consommation, s'est révélée adaptée au contexte apicole en zone rurale. Elle constitue un maillon essentiel du système, assurant la continuité du suivi en temps réel sans dépendance au réseau GSM.

Glossaire et liste des abréviations

Carte ESP32 :

L'ESP32 est une carte de développement à bas coût utilisée dans les projets électroniques et IoT. Elle intègre un microcontrôleur ESP32-WROOM-32. Ce microcontrôleur dual-core 32 bits intègre Wi-Fi, Bluetooth, des convertisseurs analogiques (ADC), numériques (GPIO). Très compacte, elle permet de connecter et piloter divers capteurs ou modules (LoRa). Facile à programmer via USB, elle est idéale pour la domotique, la télésurveillance ou la transmission de données à distance. De plus La carte est totalement compatible avec l'environnement Arduino IDE, ce qui permet de la programmer facilement avec des bibliothèques standard.

Différence entre LoRa et LoRaWan :

LoRa : La technologie physique pour transmettre des données.

LoRaWAN : Un protocole réseau basé sur LoRa, permettant de connecter des objets via des passerelles à Internet.

LPKF Laser & Electronics :

L'outil « LPKF Laser & Electronics », est une fraiseuse de gravure de carte PCB. Une machine-outil de précision utilisée pour fabriquer des circuits imprimés (PCB) en enlevant mécaniquement de la matière (généralement du cuivre) sur une plaque pré-revêtue grâce à ce que l'on appelle des forets.

Langage de programmation C :

Le langage de programmation C'est un langage de programmation généraliste, connu pour être puissant, efficace, rapide et proche du matériel, ce qui en fait un choix populaire pour les systèmes d'exploitation, les logiciels embarqués, les bases de données et les applications de haute performance.

MCP2551 :

Le MCP2551 est un transceiver CAN (Controller Area Network) utilisé pour assurer la communication entre les différentes cartes électroniques via le bus CAN, en convertissant les signaux logiques du microcontrôleur (niveau TTL) en signaux adaptés au bus CAN (niveau différentiel) et inversement.

TXD (Transmit Data) :

Broche utilisée pour transmettre des données série, généralement dans une communication UART ou CAN.

RXD (Receive Data) :

Broche utilisée pour recevoir des données série, complémentaire à TXD.

CANH (CAN High) :

Ligne haute du bus CAN utilisée pour la transmission différentielle des données.

CANL (CAN Low) :

Ligne basse du bus CAN, utilisée conjointement avec CANH pour assurer une communication robuste.

VDD (Voltage Drain Drain) :

Broche d'alimentation positive d'un circuit intégré, souvent à +5V ou +3.3V selon l'application.

VSS (Voltage Source Source) :

Référence de tension négative d'un circuit, souvent connectée à la masse (GND).

GND (Ground) :

Masse électrique du circuit, servant de référence commune pour les tensions et courants électriques.

Annexes :

Datasheet ESP 32 :

D1 R32 Board Pinout

